Ex 3: Generative Adversarial Networks (GANs) for Synthetic Image Generation

Learning Objective:

Implementation of Generative Adversarial Networks (GANs) for Synthetic Image Generation using the MNIST Dataset.

Steps:

1. Data Preparation

- **Preprocessing:** Load MNIST, scale pixels to $[-1, 1]$ for tanh compatibility, shuffle, and batch the data.

2. The Generator (The Forger)

- **Architecture:** Projects a 100D noise vector into a $7 \times 7 \times 256$ tensor, then upsamples via Conv2DTranspose to reach the final $28 \times 28 \times 1$ image size using tanh.

3. The Discriminator (The Critic)

- **Architecture:** Uses convolutional layers with LeakyReLU and Dropout to downsample the image into a single probability score via a Sigmoid activation.

4. Loss Functions

- **Objective:** Uses Binary Cross-Entropy to penalize the Discriminator for misclassifying images and the Generator for failing to "fool" the critic.

5. Optimizers

- **Setup:** Two separate Adam optimizers are used to independently update the weights of the Generator and Discriminator.

6. Training Step

- **Execution:** Noise is fed to the Generator to create fakes; the Discriminator evaluates both fakes and reals, and gradients are applied to improve both simultaneously.

7. Training Loop

- **Iteration:** The process repeats across epochs, with periodic image generation to monitor the "learning" progress of the Generator.

8. Visualization

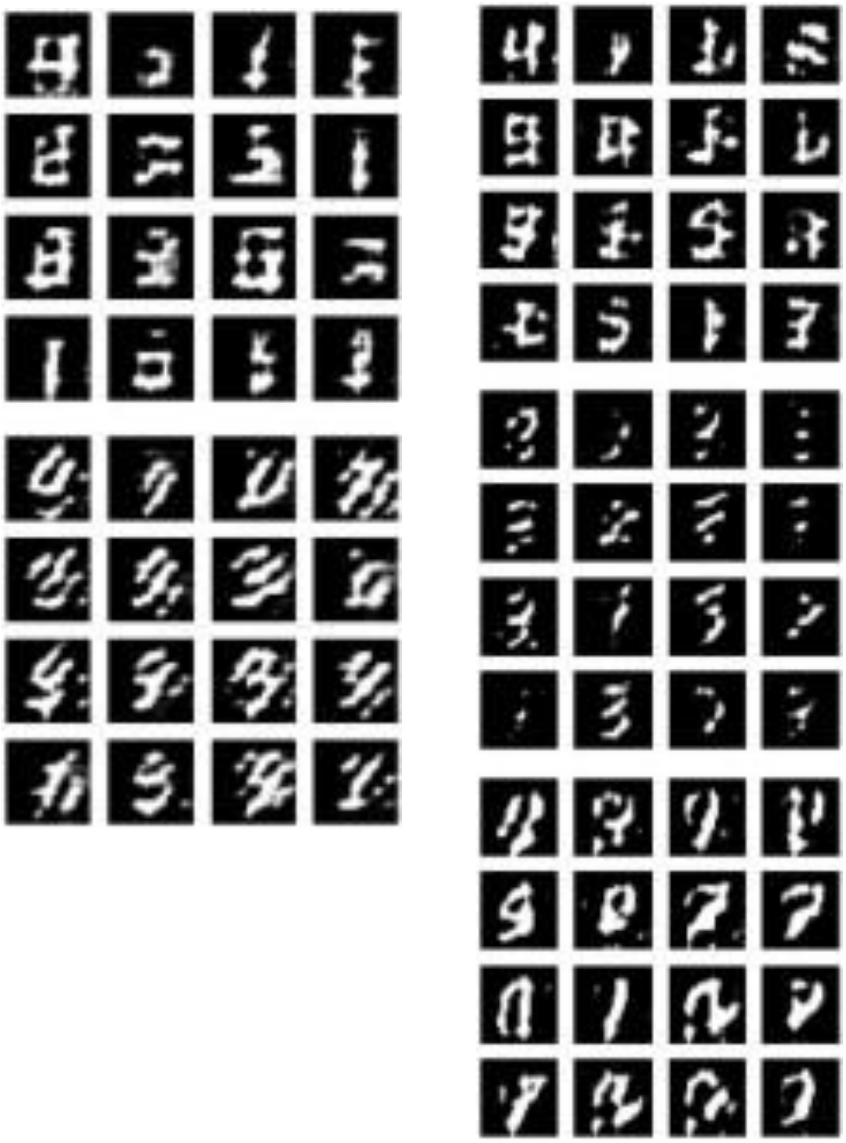
- **Comparison:** Final generated samples are plotted alongside original MNIST digits to evaluate how realistically the model has learned the data distribution.

CODE:

```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np

(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = (x_train - 127.5) / 127.5
x_train = x_train.reshape(-1,28,28,1)

BUFFER_SIZE = 60000
BATCH_SIZE = 256
dataset =
tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

def build_generator():
    model = tf.keras.Sequential()
    model.add(layers.Dense(7*7*256, input_shape=(100,)))
    model.add(layers.Reshape((7,7,256)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(128,5,strides=1,padding='same'))
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64,5,strides=2,padding='same'))
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1,5,strides=2,padding='same',activation='tanh'))
    return model

def build_discriminator():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64,5,strides=2,padding='same',input_shape=[28,28,1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128,5,strides=2,padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1,activation='sigmoid'))
    return model

generator = build_generator()
```

```
discriminator = build_discriminator()

cross_entropy = tf.keras.losses.BinaryCrossentropy()

g_optimizer = tf.keras.optimizers.Adam(0.0002)
d_optimizer = tf.keras.optimizers.Adam(0.0002)

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE,100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated_images = generator(noise)

        real_output = discriminator(images)
        fake_output = discriminator(generated_images)

        gen_loss = cross_entropy(tf.ones_like(fake_output), fake_output)

        real_loss = cross_entropy(tf.ones_like(real_output), real_output)
        fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
        disc_loss = real_loss + fake_loss

    gradients_gen = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_disc = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    g_optimizer.apply_gradients(zip(gradients_gen, generator.trainable_variables))
    d_optimizer.apply_gradients(zip(gradients_disc, discriminator.trainable_variables))

EPOCHS = 5

 for epoch in range(EPOCHS):
     for image_batch in dataset:
         train_step(image_batch)

    noise = tf.random.normal([1,100])
    generated_image = generator(noise, training=False)

    plt.imshow(generated_image[0,:,:,0], cmap='gray')
    plt.title(f"Epoch {epoch+1}")
    plt.show()
```

OUTPUT:



1. **Identify the section where MNIST dataset is loaded. Why are labels ignored?**

The dataset is loaded using:

(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

Labels are ignored because GAN is an unsupervised model that learns to generate images without needing class labels.

2. **Why are pixel values converted to [-1, 1]?**

The normalization (x_train - 127.5) / 127.5 scales pixel values to [-1, 1] to match the tanh activation output of the Generator, improving training stability.

3. **What is the purpose of the random noise vector?**

The noise vector acts as the input to the Generator, allowing it to produce diverse and random images.

4. How many Dense layers are used in the Generator?

The Generator uses 1 Dense layer at the beginning to transform the noise vector into a feature map.

5. What is the purpose of the Flatten layer?

The Flatten layer converts 2D feature maps into a 1D vector so it can be passed to the Dense layer for classification.

6. Why is sigmoid used in the final layer of the Discriminator?

Sigmoid outputs a value between 0 and 1, representing the probability of the image being real or fake.

7. What loss function is used and why?

Binary Cross Entropy is used because the problem is a binary classification task (real vs fake).

9. Why is the Discriminator trained on real and fake images?

To help it learn the difference between real images and generated (fake) images, improving its classification ability.

9. Why are fake images compared with real labels during Generator training?

The Generator tries to fool the Discriminator, so fake images are labeled as real to encourage realistic output.

10. How many synthetic images are displayed?

One image is generated per epoch. With 5 epochs, 5 images are displayed.

11. What happens if epochs increase from 20 to 50?

Generated images become clearer and more realistic, but too many epochs may lead to overfitting.

12. What happens if noise dimension is reduced from 100 to 50?

The Generator produces **less** diverse images, and quality may slightly decrease.

13. What happens if neurons increase from 128 to 256?

The Generator learns better features, often resulting in improved image quality, but training becomes slower.

14. Impact of increasing batch size from 64 to 128?

Training becomes more stable, but updates happen less frequently.

15. What happens if an extra Dense layer is added to the Generator?

It may improve feature learning, but can also lead to overfitting or instability.

16. What happens if an extra Dense layer is added to the Discriminator?

The Discriminator becomes stronger, which may make it harder for the Generator to learn.

17. Does replacing ReLU with LeakyReLU improve stability?

Yes, LeakyReLU helps prevent dead neurons and improves training stability.

18. What trend is observed in Generator and Discriminator loss?

Both losses fluctuate and oscillate, which is normal in GAN training.

19. Which model produces clearer digits: original or improved GAN?

The improved GAN produces clearer and more realistic digits.

20. Why does the Generator initially produce noisy images?

Because it starts with random weights and no training, so outputs are random.

21. What happens if the Discriminator becomes too strong?

The Generator cannot learn effectively and training may fail.

22. Why is GAN training unstable?

Because two networks compete, causing oscillations, non-convergence, and imbalance.

23. Two real-world applications of GANs (other than image generation)

Data augmentation and Image super-resolution

24. How to modify GAN to generate color images?

Change output channels to 3 and use RGB dataset:

layers.Conv2DTranspose(3, ...)

Result :

Thus, a simple Generative Adversarial Network (GAN) was implemented using TensorFlow and trained on the MNIST dataset. The Generator successfully produced synthetic handwritten digit images similar to the original MNIST digits.